

CS336 Assignment 1 实现报告

项目路径: assignment1-basics

2026 年 4 月 10 日

目录

1 Assignment 1 概述	3
1.1 完成内容、最终目的与总体结论	3
1.2 关键测试命令	3
1.3 代码结构概览	4
2 组件级实现说明	5
2.1 Tokenizer 与数据预处理	5
2.1.1 组件: <code>train_bpe</code>	5
2.1.2 组件: <code>Tokenizer</code>	6
2.1.3 组件: <code>get_batch</code>	7
2.2 基础算子与前馈网络	7
2.2.1 组件: <code>linear</code>	7
2.2.2 组件: <code>embedding</code>	8
2.2.3 组件: <code>silu</code>	8
2.2.4 组件: <code>rmsnorm</code>	8
2.2.5 组件: <code>swiglu</code>	9
2.3 数值计算、优化与训练工程	9
2.3.1 组件: <code>softmax</code>	9
2.3.2 组件: <code>cross_entropy</code>	10
2.3.3 组件: <code>gradient_clipping</code>	10
2.3.4 组件: <code>AdamW</code>	11
2.3.5 组件: <code>get_lr_cosine_schedule</code>	11
2.3.6 组件: <code>save_checkpoint</code> / <code>load_checkpoint</code>	12
2.4 注意力、位置编码与模型组装	13
2.4.1 组件: <code>scaled_dot_product_attention</code>	13
2.4.2 组件: <code>rope</code>	13
2.4.3 组件: <code>multihead_self_attention</code>	14
2.4.4 组件: <code>multihead_self_attention_with_rope</code>	14
2.4.5 组件: <code>transformer_block</code>	15
2.4.6 组件: <code>transformer_lm</code>	15

3	测试过程、定位过程与最终结果	16
3.1	测试总流程	16
3.2	关键问题的定位与修复	16
3.2.1	案例一：学习率调度 warmup 公式与测试定义不一致	16
3.2.2	案例二：BPE 训练性能瓶颈	16
3.2.3	案例三：GPT-2 参考对齐测试的可复现化	17
3.3	最终测试矩阵	17
3.4	最终汇总	18
4	关于 LLM 实现的收获与总结	19
4.1	最重要的技术收获	19
4.2	关于 LLM 的理解深化	19
4.3	总结	19

1 Assignment 1 概述

1.1 完成内容、最终目的与总体结论

本次 Assignment 1 的核心目标，是从零搭建一个 **decoder-only Transformer LM** 的最小可运行实现链条：先用 **byte-level BPE** 完成文本到 token 的映射，再实现训练阶段所需的**数据加载、损失函数、优化器、学习率调度、梯度裁剪与 checkpointing**，最后实现 **注意力、RoPE、Transformer Block 与 Transformer LM**。从工程结构上看，这个作业实际上把一个小型 LLM 训练系统拆成了若干可独立验证的组件，并要求每个组件都能在统一的测试入口上工作。

本报告聚焦于 **代码型 deliverables**。也就是说，本报告覆盖了下列 handout 中与代码直接相关的部分：

- `train_bpe, tokenizer`
- `linear, embedding, rmsnorm, positionwise_feedforward`
- `rope, softmax, scaled_dot_product_attention, multihead_self_attention`
- `transformer_block, transformer_lm`
- `cross_entropy, adamw, learning_rate_schedule, gradient_clipping`
- `data_loading, checkpointing`

最终结果是：本地完整测试集达到

47 passed, 1 xfailed

其中唯一的 `xfailed` 是测试中**显式标记的预期失败项**，并不表示实现错误。完整测试命令的 `wall-clock` 时间约为 15.04 秒，而 `pytest` 报告的测试执行时间为 13.04 秒。

1.2 关键测试命令

表 1 给出了本次实现过程中最常用的验证命令。

表 1: 关键测试命令

命令	用途
<code>./venv/bin/python -m pytest -q</code>	运行完整测试集，验证所有组件集成后的最终状态。
<code>./venv/bin/python -m pytest -k test_train_bpe_speed</code>	专门验证 BPE 训练的速度约束。
<code>./venv/bin/python -m pytest -k test_get_lr_cosine_schedule</code>	验证 warmup + cosine decay 学习率调度。
<code>./venv/bin/python -m pytest -k 'test_rope or test_multihead_self_attention_with_rope'</code>	验证 RoPE 与带 RoPE 的 MHA。
<code>./venv/bin/python -m pytest -k 'test_transformer_block or test_transformer_lm'</code>	验证 Block 与 LM 级别的集成正确性。
<code>./venv/bin/python -m py_compile ...</code>	在运行单测前先做语法级静态检查。

1.3 代码结构概览

最终代码主要位于 `cs336_basics` 目录中，形成了如下功能分层：

- **Tokenizer 层**: `tokenizer.py`
- **基础张量算子**: `linear.py`, `embedding.py`, `silu.py`, `rmsnorm.py`, `swiglu.py`
- **注意力层**: `softmax.py`, `scaled_dot_product_attention.py`, `rope.py`, `multihead_self_attention.py`, `multihead_self_attention_with_rope.py`
- **模型组装层**: `transformer_block.py`, `transformer_lm.py`
- **训练工具层**: `get_batch.py`, `cross_entropy.py`, `gradient_clipping.py`, `AdamW.py`, `lr_schedule.py`, `save_load_checkpoint.py`

2 组件级实现说明

2.1 Tokenizer 与数据预处理

2.1.1 组件: train_bpe

1. 作用、输入与输出 该组件用于从原始文本语料训练 **byte-level BPE**。输入为文本路径 `input_path`、目标词表大小 `vocab_size` 与可选的 `special_tokens`；输出为

$$\text{vocab} : \{\text{id} \mapsto \text{bytes}\}, \quad \text{merges} : [(b_1, b_2), \dots]$$

其中 `vocab` 是 token id 到字节串的映射，`merges` 是按学习顺序记录的 merge 规则。

2. 代码实现摘要 核心代码如下：

```
token_counts = _collect_pretoken_counts(text, normalized_special_tokens)
pair_counts, pair_to_sequences = _build_pair_statistics(token_counts)

while len(vocab) < vocab_size:
    best_pair = max(pair_counts.items(), key=lambda item: (item[1], item[0]))[0]
    affected_sequences = list(pair_to_sequences.get(best_pair, ()))
    ...
```

实现流程是：

1. 用 GPT-2 风格 regex 做 pre-tokenization；
2. 将每个 pretoken 转为 UTF-8 字节序列；
3. 统计所有相邻 pair 的频次；
4. 反复选出最高频 pair 做 merge；
5. 更新词表与 merge 序列，直到达到目标词表大小。

3. 数学原理 如果把某个 pretoken 写成 token 序列

$$t = (u_1, u_2, \dots, u_n),$$

则 BPE 在每一轮选择

$$(u^*, v^*) = \arg \max_{(u, v)} (c(u, v), (u, v)),$$

其中 $c(u, v)$ 是 pair (u, v) 在整个语料中的总频次；当频次并列时，再按 pair 的字典序做 tie-break。选中后执行

$$u^*, v^* \longrightarrow u^*v^*.$$

代码中的

```
best_pair = max(pair_counts.items(), key=lambda item: (item[1], item[0]))[0]
```

正对应这个规则。

4. 工程技巧 这一部分用了最重要的工程优化：**增量维护 pair 统计**。初版做法是在每一轮 merge 后对全部 token sequence 重新统计 pair 频次，复杂度较高，导致 `test_train_bpe_speed` 失败。最终版本维护了：

- `pair_counts`: 全局 pair 频次；
- `pair_to_sequences`: pair 到包含该 pair 的 sequence 的反向索引。

这样每轮只更新受到当前 `best_pair` 影响的序列，而不必全量重扫整个语料。

2.1.2 组件: Tokenizer

1. 作用、输入与输出 Tokenizer 负责在文本与 token id 序列之间建立双向映射。输入包括：

- `vocab`: `dict[int, bytes]`
- `merges`: `list[tuple[bytes, bytes]]`
- `special_tokens`: `Sequence[str] | None`

输出行为则由以下接口提供：

- `encode(text)`: `str → list[int]`
- `decode(ids)`: `list[int] → str`
- `encode_iterable(iterable)`: 流式编码文本块
- `from_files(...)`: 从 GPT-2 风格词表文件恢复 tokenizer

2. 代码实现摘要 `encode` 的核心结构是：

```
for is_special, unit in iter_text_units(text, self.special_tokens):
    if is_special:
        token_ids.append(self.special_token_to_id[unit])
    else:
        token_ids.extend(self._encode_pretoken_to_ids(unit))
```

`decode` 的核心结构是：

```
token_bytes = [self.vocab[token_id] for token_id in ids]
return b"".join(token_bytes).decode("utf-8", errors="replace")
```

`encode_iterable` 则维护一个 buffer，在 chunk 边界上保留不稳定尾部，确保跨 chunk 的 pretoken / special token 不被错误拆开。

3. 数学原理 编码过程本质上是复合映射：

$$\text{text} \xrightarrow{\text{special split}} \text{units} \xrightarrow{\text{pretokenize}} \text{pretokens} \xrightarrow{\text{BPE merge}} \text{byte tokens} \xrightarrow{\text{lookup}} \text{ids}.$$

解码过程则是

$$(i_1, i_2, \dots, i_n) \xrightarrow{\text{vocab lookup}} (b_1, b_2, \dots, b_n) \xrightarrow{\text{concat}} b \xrightarrow{\text{utf-8 decode}} \text{text}.$$

4. 工程技巧 这里用了两点工程技巧：

- **重叠 special token 的长度优先匹配**：通过按长度降序排序 special token，解决短 token 抢先匹配的问题；
- **流式编码的稳定尾部分离**：encode_iterable 不会简单地对每个 chunk 直接编码，而是保留 chunk 末尾可能跨边界延续的 pretoken。

2.1.3 组件：get_batch

1. 作用、输入与输出 get_batch 从一维 token 序列中随机抽取训练样本。输入是

dataset, batch_size, context_length, device,

输出是两个 shape 都为

(batch_size, context_length)

的 tensor：

X, Y .

2. 代码实现摘要

```
start_indices = np.random.randint(0, num_possible_starts, size=batch_size, dtype=np.int64)
offsets = np.arange(context_length + 1, dtype=np.int64)
batch_np = dataset[start_indices[:, None] + offsets[None, :]]
x = batch[:, :-1]
y = batch[:, 1:]
```

3. 数学原理 设一维 token 流为

$$(x_0, x_1, \dots, x_{N-1}),$$

随机抽取起点 s_i 后，定义：

$$X_i = (x_{s_i}, x_{s_i+1}, \dots, x_{s_i+L-1}),$$

$$Y_i = (x_{s_i+1}, x_{s_i+2}, \dots, x_{s_i+L}),$$

其中 $L = \text{context_length}$ 。这就是标准 next-token prediction 训练对。

4. 工程技巧 实现里一次性取出长度 $L+1$ 的窗口块，然后再切成 X 与 Y 。这样只做一次 gather；同时 CUDA 路径下使用了 pin_memory 与 non_blocking=True。

2.2 基础算子与前馈网络

2.2.1 组件：linear

1. 作用、输入与输出 linear 实现线性层。输入是权重

$$W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$$

和特征

$$X \in \mathbb{R}^{\cdots \times d_{\text{in}}},$$

输出

$$Y \in \mathbb{R}^{\cdots \times d_{\text{out}}}.$$

2. 代码实现摘要

```
return in_features @ weights.transpose(-1, -2)
```

3. 数学原理

$$Y = XW^{\top}.$$

因为代码中 `weights` 的存储形式是 $(d_{\text{out}}, d_{\text{in}})$ ，所以在右乘前需要转置。

2.2.2 组件: embedding

1. 作用、输入与输出 `embedding` 根据 token id 取词向量。输入是

$$W \in \mathbb{R}^{V \times d_{\text{model}}}, \quad \text{token_ids} \in \mathbb{N}^{\cdots}$$

输出

$$E \in \mathbb{R}^{\cdots \times d_{\text{model}}}.$$

2. 代码实现摘要

```
return weights[token_ids]
```

3. 数学原理 这是典型的查表操作：

$$E_i = W[\text{token_id}_i].$$

2.2.3 组件: silu

1. 作用、输入与输出 逐元素非线性激活，输入输出 shape 完全一致。

2. 代码实现摘要

```
return in_features * torch.sigmoid(in_features)
```

3. 数学原理

$$\text{SiLU}(x) = x \cdot \sigma(x), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

2.2.4 组件: rmsnorm

1. 作用、输入与输出 对最后一个维度执行 RMSNorm。输入是特征 x 与可学习缩放参数 w ，输出与输入同 shape。

2. 代码实现摘要

```
rms = torch.sqrt(torch.mean(in_features * in_features, dim=-1, keepdim=True) + eps)
normalized = in_features / rms
return normalized * weights
```

3. 数学原理 对单个向量 $x \in \mathbb{R}^d$, RMSNorm 定义为

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}} \odot w.$$

代码中的 `torch.mean(in_features * in_features, dim=-1)` 对应分母里的均方, 随后再乘以 `weights` 完成仿射缩放。

2.2.5 组件: swiglu

1. 作用、输入与输出 SwiGLU 是 Transformer FFN 的核心前馈模块。输入是

$$X \in \mathbb{R}^{\cdots \times d_{\text{model}}},$$

输出仍然是

$$\mathbb{R}^{\cdots \times d_{\text{model}}}.$$

2. 代码实现摘要

```
w1_out = linear(..., weights=w1_weight, in_features=in_features)
w3_out = linear(..., weights=w3_weight, in_features=in_features)
gated = silu(w1_out) * w3_out
return linear(..., weights=w2_weight, in_features=gated)
```

3. 数学原理

$$\text{SwiGLU}(x) = W_2(\text{SiLU}(W_1 x) \odot (W_3 x)).$$

其中 W_1 与 W_3 都把输入从 d_{model} 映射到 d_{ff} , 而 W_2 再把中间结果映射回 d_{model} 。

2.3 数值计算、优化与训练工程

2.3.1 组件: softmax

1. 作用、输入与输出 沿指定维度 `dim` 把任意 logits 转成概率分布, 输出 `shape` 与输入完全一致。

2. 代码实现摘要

```
max_values = torch.amax(in_features, dim=dim, keepdim=True)
shifted = in_features - max_values
exp_shifted = torch.exp(shifted)
partition = torch.sum(exp_shifted, dim=dim, keepdim=True)
return exp_shifted / partition
```

3. 数学原理

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

为避免数值溢出，代码先做平移：

$$\tilde{x}_i = x_i - \max_j x_j.$$

由于 softmax 对所有分量同时减去同一个常数不变，因此

$$\text{softmax}(x) = \text{softmax}(\tilde{x}).$$

4. 工程技巧 这里的关键技巧是**减去最大值**，这是数值稳定版 softmax 的标准写法。

2.3.2 组件：cross_entropy

1. 作用、输入与输出 对一个 batch 的 logits 与离散标签计算平均交叉熵。输入：

$$Z \in \mathbb{R}^{B \times V}, \quad y \in \{0, \dots, V-1\}^B.$$

输出是标量 loss。

2. 代码实现摘要

```
max_values = torch.amax(inputs, dim=-1, keepdim=True)
shifted = inputs - max_values
log_sum_exp = torch.log(torch.sum(torch.exp(shifted), dim=-1)) + max_values.squeeze(-1)
target_logits = inputs.gather(dim=-1, index=targets.unsqueeze(-1)).squeeze(-1)
losses = log_sum_exp - target_logits
return losses.mean()
```

3. 数学原理 单样本交叉熵可写成

$$\mathcal{L}(z, y) = -\log \frac{e^{z_y}}{\sum_j e^{z_j}} = \log \sum_j e^{z_j} - z_y.$$

代码中的 log_sum_exp 对应前一项，target_logits 对应后一项。

4. 工程技巧 依然采用了 **log-sum-exp 的稳定写法**，避免直接对大 logits 做指数运算。

2.3.3 组件：gradient_clipping

1. 作用、输入与输出 对一组参数梯度做 global L_2 norm clipping，输入是一组参数与阈值 max_l2_norm，输出为空，但会**原地修改** param.grad。

2. 代码实现摘要

```
params_with_grad = [p for p in parameters if p.grad is not None]
total_norm = torch.norm(
    torch.stack([torch.norm(p.grad.detach(), p=2) for p in params_with_grad]),
    p=2,
)
clip_coef = max_l2_norm / (total_norm + 1e-6)
```

```
if clip_coef < 1:
    for param in params_with_grad:
        param.grad.mul_(clip_coef)
```

3. 数学原理 设所有梯度为 $g_1, \dots, g_n$, 则全局范数为

$$\|g\|_2 = \sqrt{\sum_{i=1}^n \|g_i\|_2^2}.$$

若 $\|g\|_2 > \tau$, 则统一缩放:

$$g_i \leftarrow g_i \cdot \frac{\tau}{\|g\|_2}.$$

4. 工程技巧 使用 `mul_` 做原地更新, 避免无意义的新 tensor 分配。

2.3.4 组件: AdamW

1. 作用、输入与输出 实现带 decoupled weight decay 的 AdamW 优化器, 输入是一组模型参数与超参数 $(lr, \beta_1, \beta_2, \epsilon, \lambda)$ 。核心输出是一次 `step()` 后的参数更新。

2. 代码实现摘要

```
exp_avg.mul_(beta1).add_(grad, alpha=1.0 - beta1)
exp_avg_sq.mul_(beta2).addcmul_(grad, grad, value=1.0 - beta2)
bias_correction1 = 1.0 - beta1**step
bias_correction2 = 1.0 - beta2**step
p.data.mul_(1.0 - lr * weight_decay)
p.data.addcddiv_(corrected_exp_avg, denom, value=-lr)
```

3. 数学原理 AdamW 的状态更新为

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

偏置修正为

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

最终参数更新为

$$\theta \leftarrow \theta(1 - lr \cdot \lambda) - lr \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

其中第一项就是 **decoupled weight decay**。

4. 工程技巧 实现复用了 PyTorch Optimizer 基类的 `param_groups` 与 `state` 机制, 使接口与官方优化器一致。

2.3.5 组件: get_lr_cosine_schedule

1. 作用、输入与输出 根据当前 iteration `it` 返回学习率。输入包括最大/最小学习率、warmup 步数和 cosine 衰减总长度, 输出是当前步的标量学习率。

2. 代码实现摘要

```
if warmup_iters > 0 and it < warmup_iters:
    return max_learning_rate * it / warmup_iters
if it >= cosine_cycle_iters:
    return min_learning_rate
progress = (it - warmup_iters) / (cosine_cycle_iters - warmup_iters)
cosine_decay = 0.5 * (1.0 + math.cos(math.pi * progress))
```

3. 数学原理 该 schedule 是分段函数：

$$\alpha_t = \begin{cases} \alpha_{\max} \cdot \frac{t}{T_w}, & 0 \leq t < T_w, \\ \alpha_{\min} + \frac{1 + \cos(\pi \cdot p_t)}{2} (\alpha_{\max} - \alpha_{\min}), & T_w \leq t < T_c, \\ \alpha_{\min}, & t \geq T_c, \end{cases}$$

其中

$$p_t = \frac{t - T_w}{T_c - T_w}.$$

4. 工程技巧 这里最关键的并不是代码量，而是**边界定义与测试语义一致**：第 0 步学习率应为 0，而不是 $\alpha_{\max}/T_w$ 。

2.3.6 组件：save_checkpoint / load_checkpoint

1. 作用、输入与输出 保存并恢复训练状态。输入包括模型、优化器与 iteration，输出要么写入文件，要么恢复状态并返回被恢复的 iteration。

2. 代码实现摘要

```
checkpoint = {
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "iteration": iteration,
}
torch.save(checkpoint, out)
```

加载时：

```
checkpoint = torch.load(src, map_location="cpu")
model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
return checkpoint["iteration"]
```

4. 工程技巧 这里采用 PyTorch 标准的 state_dict 协议，优点是和模型/优化器内部结构解耦。

2.4 注意力、位置编码与模型组装

2.4.1 组件: scaled_dot_product_attention

1. 作用、输入与输出 给定

$$Q \in \mathbb{R}^{\cdots \times n_q \times d_k}, \quad K \in \mathbb{R}^{\cdots \times n_k \times d_k}, \quad V \in \mathbb{R}^{\cdots \times n_k \times d_v},$$

输出

$$O \in \mathbb{R}^{\cdots \times n_q \times d_v}.$$

mask=True 表示允许注意力流动, False 表示该位置必须被屏蔽。

2. 代码实现摘要

```
scores = torch.matmul(Q, K.transpose(-1, -2)) / math.sqrt(d_k)
if mask is not None:
    scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)
attention_probs = softmax(scores, dim=-1)
return torch.matmul(attention_probs, V)
```

3. 数学原理

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

mask 的实现方式是在 softmax 前把非法位置加上 $-\infty$, 即:

$$S_{ij} = \begin{cases} \frac{q_i^\top k_j}{\sqrt{d_k}}, & M_{ij} = \text{True}, \\ -\infty, & M_{ij} = \text{False}. \end{cases}$$

4. 工程技巧 注意力实现支持任意数量的 **batch-like 维度**, 例如 $(batch, heads, seq, d)$ 。

2.4.2 组件: rope

1. 作用、输入与输出 RoPE 对 Query 或 Key 的最后一个维度做位置相关旋转, 输入输出 shape 相同。

2. 代码实现摘要

```
x_even = in_query_or_key[..., ::2]
x_odd = in_query_or_key[..., 1::2]
rotated_even = x_even * cos - x_odd * sin
rotated_odd = x_even * sin + x_odd * cos
```

3. 数学原理 把维度两两分组, 记第 m 对维度对应角度为

$$\phi_m(p) = \frac{p}{\Theta^{2m/d_k}},$$

则对位置 p 上的向量分量 (x_{2m}, x_{2m+1}) 做二维旋转:

$$\begin{pmatrix} x'_{2m} \\ x'_{2m+1} \end{pmatrix} = \begin{pmatrix} \cos \phi_m(p) & -\sin \phi_m(p) \\ \sin \phi_m(p) & \cos \phi_m(p) \end{pmatrix} \begin{pmatrix} x_{2m} \\ x_{2m+1} \end{pmatrix}.$$

4. 工程技巧 在多头注意力中，**head 维度被当作 batch-like 维度处理**，因此同一套旋转公式可以直接广播到所有 head。

2.4.3 组件: multihead_self_attention

1. 作用、输入与输出 实现因果多头自注意力。输入为

$$X \in \mathbb{R}^{\dots \times T \times d_{\text{model}}},$$

输出同样是

$$\mathbb{R}^{\dots \times T \times d_{\text{model}}}.$$

2. 代码实现摘要

```
q = linear(..., weights=q_proj_weight, in_features=in_features)
k = linear(..., weights=k_proj_weight, in_features=in_features)
v = linear(..., weights=v_proj_weight, in_features=in_features)
q = q.reshape(..., num_heads, q_head_dim).transpose(-3, -2)
...
causal_mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool, device=
    in_features.device))
attention_output = scaled_dot_product_attention(Q=q, K=k, V=v, mask=causal_mask)
```

3. 数学原理

$$\text{MHA}(x) = W_O \cdot \text{Concat}(\text{head}_1, \dots, \text{head}_h),$$

其中每个 head 是

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i).$$

因果约束要求 query 位置 i 只能看见 $j \leq i$ 的 key，因此 mask 是下三角矩阵。

4. 工程技巧 Q/K/V 的投影被实现为三个 **batched 线性变换**，而不是按 head 循环；这能更好利用张量并行。

2.4.4 组件: multihead_self_attention_with_rope

1. 作用、输入与输出 它是普通因果 MHA 的 RoPE 扩展版本。与上一个组件不同的是：**只对 Q 和 K 应用 RoPE，不对 V 应用**。

2. 代码实现摘要

```
q = rope(..., in_query_or_key=q, token_positions=token_positions)
k = rope(..., in_query_or_key=k, token_positions=token_positions)
attention_output = scaled_dot_product_attention(Q=q, K=k, V=v, mask=causal_mask)
```

3. 数学原理

$$Q' = \text{RoPE}(Q), \quad K' = \text{RoPE}(K), \quad \text{Attention}(Q', K', V).$$

这会让注意力分数显式携带相对位置信息。

2.4.5 组件: transformer_block

1. 作用、输入与输出 实现一个 **pre-norm Transformer block**。输入输出 shape 都是

$$\mathbb{R}^{batch \times seq \times d_{model}}.$$

2. 代码实现摘要

```
x_norm = rmsnorm(..., weights=weights["ln1.weight"], in_features=in_features)
attn_out = multihead_self_attention_with_rope(..., in_features=x_norm, ...)
residual = in_features + attn_out
residual_norm = rmsnorm(..., weights=weights["ln2.weight"], in_features=residual)
ff_out = swiglu(..., in_features=residual_norm)
return residual + ff_out
```

3. 数学原理 令输入为 x , 则 block 的两个子层分别是:

$$y = x + \text{MHA}(\text{RMSNorm}(x)),$$

$$z = y + \text{FFN}(\text{RMSNorm}(y)).$$

这里 FFN 实际上就是前面定义的 SwiGLU。

4. 工程技巧 采用 **pre-norm** 结构, 而不是 **post-norm**。这样梯度路径更稳定, 也更符合 handout 的要求。

2.4.6 组件: transformer_lm

1. 作用、输入与输出 这是最终的语言模型前向传播。输入是 token id 张量

$$I \in \mathbb{N}^{B \times T},$$

输出是 logits

$$Z \in \mathbb{R}^{B \times T \times V}.$$

2. 代码实现摘要

```
hidden = embedding(..., weights=weights["token_embeddings.weight"], token_ids=in_indices)
for layer_idx in range(num_layers):
    layer_weights = {...}
    hidden = transformer_block(..., weights=layer_weights, in_features=hidden)
hidden = rmsnorm(..., weights=weights["ln_final.weight"], in_features=hidden)
return linear(..., weights=weights["lm_head.weight"], in_features=hidden)
```

3. 数学原理 定义

$$H_0 = \text{Embed}(I),$$

第 ℓ 层 block 为 B_ℓ , 则

$$H_{\ell+1} = B_\ell(H_\ell), \quad \ell = 0, \dots, L-1.$$

最后输出

$$Z = H_L^{(\text{final norm})} W_{\text{lm}}^\top.$$

3 测试过程、定位过程与最终结果

3.1 测试总流程

测试策略采用由局部到整体的方式：

1. 先通过 `py_compile` 排除语法问题；
2. 然后按组件粒度运行单测，例如 `test_silu`、`test_rmsnorm`、`test_rope`；
3. 对关键性能约束单独验证，如 `test_train_bpe_speed`；
4. 最后运行完整测试集 `pytest -q` 做集成回归。

3.2 关键问题的定位与修复

3.2.1 案例一：学习率调度 warmup 公式与测试定义不一致

现象 `test_get_lr_cosine_schedule` 初次失败，失败用例显示期望序列的开头是

`[0, 0.142857, 0.285714, ...]`

而初版实现把 warmup 写成了

$$\alpha_t = \alpha_{\max} \cdot \frac{t+1}{T_w},$$

因此第 0 步学习率不是 0，而是 $\alpha_{\max}/T_w$ 。

定位 通过直接阅读 `tests/test_optimizer.py` 中的 `expected_lrs`，可以明确看出测试采用的是

$$\alpha_t = \alpha_{\max} \cdot \frac{t}{T_w}.$$

因此问题不是余弦退火部分，而是 warmup 起点定义。

修复 将 warmup 段修改为：

```
if warmup_iters > 0 and it < warmup_iters:
    return max_learning_rate * it / warmup_iters
```

修复后，单测 `test_get_lr_cosine_schedule` 通过。

3.2.2 案例二：BPE 训练性能瓶颈

现象 `test_train_bpe_speed` 初次失败，测试内部记录到的训练时间约为 2.17 秒，而阈值是 1.5 秒。

定位 初版 `train_bpe` 每一轮都执行：

1. 对全部 token sequence 重新统计 pair 频次；
2. 对全部 token sequence 重新执行 merge。

这使得复杂度近似变成“merge 轮数 × 全语料重扫”，在小语料上也会明显超时。

修复 最终版本改成了**增量统计**：

- 用 `pair_counts` 保存全局 `pair` 频次；
- 用 `pair_to_sequences` 保存 `pair` 到 `sequence` 的反向索引；
- 每轮只更新包含 `best_pair` 的受影响序列。

修复后，`test_train_bpe_speed` 通过。单独运行该测试命令时，`pytest` 报告时间为 0.31 秒；包含解释器与 `pytest` 启动开销的 `wall-clock` 时间约为 1.94 秒，但测试内部计时已经低于 1.5 秒阈值。

3.2.3 案例三：GPT-2 参考对齐测试的可复现化

现象 `tokenizer` 对齐测试依赖 `tiktoken.get_encoding("gpt2")`。为了让对齐测试在项目内可复现，需要保证 GPT-2 参考资源 `encoder.json` 与 `vocab.bpe` 可以被本地稳定读取。

处理方式 最终方案是在 `tests/adapters.py` 顶部初始化本地 `TIKTOKEN_CACHE_DIR`，并把项目内已有的 GPT-2 fixture 写入缓存。其中：

- `encoder.json` 直接使用 `tests/fixtures/gpt2_vocab.json`
- `vocab.bpe` 使用 `tests/fixtures/gpt2_merges.txt`，并补写 `#version: 0.2` 头

这样对齐测试比较的是**实现行为**，而不是外部资源可达性。

3.3 最终测试矩阵

表 2 总结了最终测试结果。所有测试均通过，只有 `test_encode_memory_usage` 是测试中显式标记的 `xfail`，属于预期行为。

表 2: 最终测试矩阵

测试名	结果	说明
<code>test_get_batch</code>	PASS	数据采样、右移标签与 <code>device</code> 行为正确。
<code>test_linear</code>	PASS	线性层矩阵乘法与 <code>shape</code> 对齐。
<code>test_embedding</code>	PASS	Embedding 查表正确。
<code>test_swiglu</code>	PASS	前馈门控结构与参考权重对齐。
<code>test_scaled_dot_product_attention</code>	PASS	2D attention 正确。
<code>test_4d_scaled_dot_product_attention</code>	PASS	batch-like 维度广播正确。
<code>test_multihead_self_attention</code>	PASS	因果 MHA 正确。
<code>test_multihead_self_attention_with_rope</code>	PASS	带 RoPE 的 MHA 正确。
<code>test_transformer_lm</code>	PASS	完整 LM 前向通过。
<code>test_transformer_lm_truncated_input</code>	PASS	截断输入下的 LM 行为正确。
<code>test_transformer_block</code>	PASS	pre-norm Block 正确。
<code>test_rmsnorm</code>	PASS	RMSNorm 数值对齐。
<code>test_rope</code>	PASS	位置旋转编码正确。
<code>test_silu_matches_pytorch</code>	PASS	SiLU 与 PyTorch 对齐。
<code>test_softmax_matches_pytorch</code>	PASS	softmax 数值稳定且正确。
<code>test_cross_entropy</code>	PASS	交叉熵与参考对齐。
<code>test_gradient_clipping</code>	PASS	global norm clipping 行为正确。

测试名	结果	说明
test_adamw	PASS	AdamW 与参考/官方实现对齐。
test_get_lr_cosine_schedule	PASS	warmup + cosine 衰减正确。
test_checkpointing	PASS	checkpoint 存取一致。
test_roundtrip_empty	PASS	空字符串 roundtrip 正确。
test_empty_matches_tiktoken	PASS	空输入与 GPT-2 参考一致。
test_roundtrip_single_character	PASS	单字符 roundtrip 正确。
test_single_character_matches_tiktoken	PASS	单字符与参考一致。
test_roundtrip_single_unicode_character	PASS	单 Unicode 字符 roundtrip 正确。
test_single_unicode_character_matches_tiktoken	PASS	单 Unicode 字符与参考一致。
test_roundtrip_ascii_string	PASS	ASCII 串 roundtrip 正确。
test_ascii_string_matches_tiktoken	PASS	ASCII 串与参考一致。
test_roundtrip_unicode_string	PASS	Unicode 串 roundtrip 正确。
test_unicode_string_matches_tiktoken	PASS	Unicode 串与参考一致。
test_roundtrip_unicode_string_with_special_tokens	PASS	特殊 token 混合输入 roundtrip 正确。
test_unicode_string_with_special_tokens_matches_tiktoken	PASS	与参考一致。
test_overlapping_special_tokens	PASS	重叠 special token 匹配正确。
test_address_roundtrip	PASS	地址文本 roundtrip 正确。
test_address_matches_tiktoken	PASS	地址文本与参考一致。
test_german_roundtrip	PASS	德文文本 roundtrip 正确。
test_german_matches_tiktoken	PASS	德文文本与参考一致。
test_tinystories_sample_roundtrip	PASS	TinyStories 样本 roundtrip 正确。
test_tinystories_matches_tiktoken	PASS	TinyStories 样本与参考一致。
test_encode_special_token_trailing_newlines	PASS	特殊 token 邻接换行的行为正确。
test_encode_special_token_double_newline_non_whitespace	PASS	特殊 token 后双换行且后接文本时行为正确。
test_encode_iterable_tinystories_sample_roundtrip	PASS	流式编码 roundtrip 正确。
test_encode_iterable_tinystories_matches_tiktoken	PASS	流式编码与参考一致。
test_encode_iterable_memory_usage	PASS	流式编码满足内存约束。
test_encode_memory_usage	XFAIL	预期失败: <code>encode</code> 不是内存优化接口。
test_train_bpe_speed	PASS	BPE 训练速度满足阈值。
test_train_bpe	PASS	BPE merges / vocab 正确。
test_train_bpe_special_tokens	PASS	特殊 token 边界处理正确。

3.4 最终汇总

最终完整测试命令

```
./venv/bin/python -m pytest -q
```

得到的最终结果是

```
47 passed, 1 xfailed in 13.04s
```

包含解释器与 pytest 启动开销的 wall-clock 时间约为

15.04 s.

4 关于 LLM 实现的收获与总结

4.1 最重要的技术收获

1. **LLM 并不是一个单独的“模型文件”，而是完整的系统链路。**

从 tokenizer、batch 构造、损失函数、优化器到注意力与 Block 组装，任何一层出错都可能让整体训练失败。

2. **工程正确性与数学正确性同等重要。**

例如 softmax 与 cross_entropy 的数学公式都很短，但如果不做数值稳定处理，实际训练中会立刻出问题。

3. **性能问题不能只靠“更快的机器”掩盖。**

train_bpe 的主要瓶颈来自算法结构，而不是单条指令慢。把“全量重扫”改成“增量维护索引”以后，性能才真正改善。

4. **自注意力的核心难点在 shape 管理，而不是公式本身。**

公式

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

并不复杂，真正容易错的是 head 拆分、batch-like 维度广播、causal mask 的形状，以及 RoPE 应该施加在 Q/K 而不是 V 上。

5. **Transformer 的稳定训练依赖若干“局部正确的组件约定”。**

比如 pre-norm、global gradient clipping、AdamW 的 decoupled weight decay、cosine LR schedule 的 warmup 边界，都是看似局部但对整体训练质量有直接影响的设计。

4.2 关于 LLM 的理解深化

通过手写这一版 Assignment 1，我对 LLM 的理解从“知道结构”进一步推进到了“知道数据和张量是怎样真正流动的”。例如，过去在阅读论文时，RoPE、MHA、RMSNorm 往往被视作已经打包好的模块；但在这次实现中，必须亲自处理如下事实：

- tokenization 实际上决定了后续模型要学习的离散表示空间；
- attention 的 mask 语义、softmax 的数值稳定性，会直接影响训练是否可靠；
- Block 与 LM 的实现不是“搭积木”那么简单，而是要求每一层 shape 都严格一致；
- 训练系统的可恢复性 (checkpoint)、可控性 (lr schedule)、稳定性 (gradient clipping) 和模型前向本身同样重要。

4.3 总结

从结果上看，这次 Assignment 1 完成了一个从 tokenizer 到 Transformer LM 前向、从 batch 到 optimizer step 的完整最小闭环。如果只从代码规模看，这些函数都不长；但真正困难的部分在于：

1. 把 handout 中的数学定义精确地落实为张量操作；
2. 保证所有中间接口（shape、dtype、mask 语义、special token 边界）完全一致；
3. 用测试驱动的方式把 correctness、numerical stability 和 performance 都收口。

因此，这份作业最有价值的地方，并不只是“写出了一个小 Transformer”，而是把 **LLM 系统实现中的关键思想、关键接口和关键调试方法**都走了一遍。对后续继续做训练脚本、解码、实验日志和更大规模实验来说，这一版代码已经形成了一个可靠的基础。